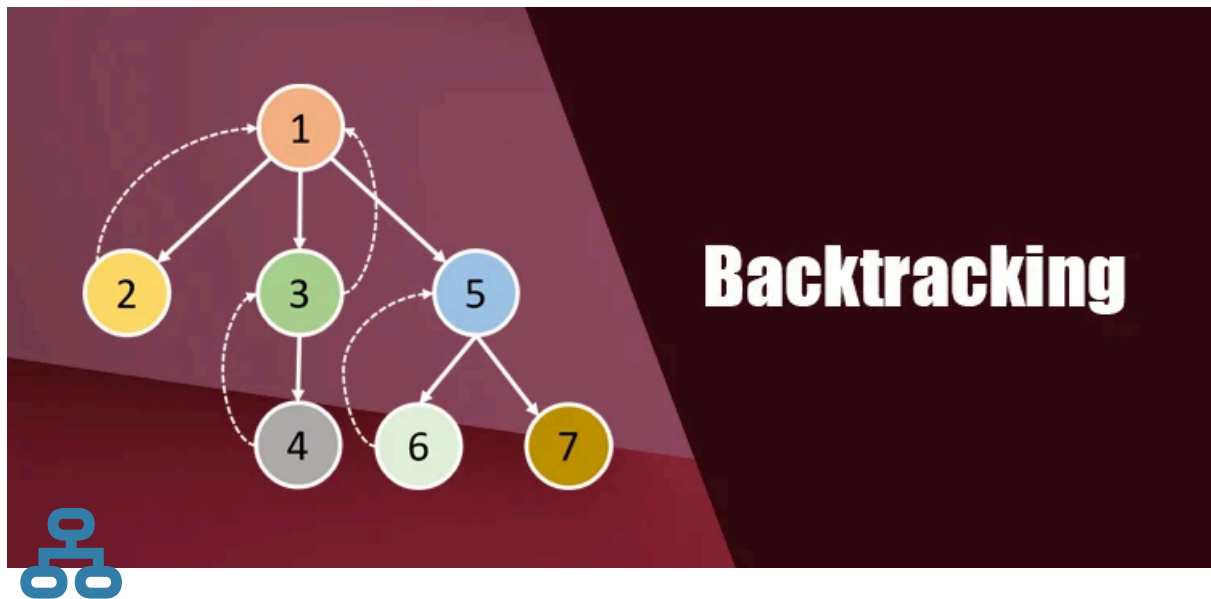




3. Backtracking

"When you reach a dead-end, return to the last decision"

Previous Section:

 2. Divide and Conquer

Contents:

1. What is Backtracking?

2. Implementation of Backtracking

Crossword Game

Maze Game

Can Backtracking Find the Shortest Path?

3. Applications of Backtracking

4. Advantages of Backtracking

5. Disadvantages of Backtracking

Summary

References

Some problems are not about finding the *fastest* path or the *smallest* value. Sometimes, the real challenge is simply this:

| ***“Out of all possible choices... which one actually works?”***

And that is exactly where Backtracking appears. Backtracking is a problem-solving technique built around exploration. Instead of following one fixed route, the algorithm continuously tries different possibilities, step by step, searching for a valid solution.

You can think of it like walking through a maze. At every intersection, you choose a direction and continue forward. If the path eventually leads to a wall or a dead-end, you do not restart from the beginning. Instead, you return to the last decision point, undo the wrong choice, and try another path. That process of trying a choice, checking whether it works, and undoing it if it fails, is the core idea behind Backtracking.

Unlike optimization algorithms that mainly focus on improving a value, Backtracking focuses on systematically exploring possibilities. In many cases, it may generate multiple candidate solutions before finally finding the correct one—or even all possible solutions.

This makes Backtracking extremely useful for problems involving permutations, combinations, puzzles, path finding, and constraint-based search problems.

Common examples include: Sudoku solvers; Maze traversal; Crossword puzzles; N-Queens problems; Generating permutations

At its heart, Backtracking follows one simple philosophy:

| ***“If a choice leads nowhere... go back and try something else.”***

1. What is Backtracking?

Backtracking is an algorithmic technique that solves problems by exploring possible choices recursively. It works by building a solution step by step and checking whether the current path can still lead to a valid answer.

If the current choice eventually becomes invalid, the algorithm immediately abandons that path, returns to the previous state, and tries another possibility.

This is why it is called **Backtracking**: The algorithm moves forward while searching, but moves backward whenever a path fails.

The process usually follows this pattern:

- Make a choice → Continue exploring recursively → Check whether the current path is valid → If it fails, undo the choice → Try another option

This **“choose → explore → undo”** cycle continues until: a valid solution is found, or every possibility has been exhausted.

Backtracking is heavily connected to recursion because each new choice creates a smaller version of the same problem. The algorithm repeatedly calls itself while exploring deeper possibilities.

A useful way to think about Backtracking is this:

Divide and Conquer splits a problem into independent subproblems. Backtracking explores decisions one by one while continuously checking whether the current path is still possible.

Backtracking problems are commonly divided into three categories:

- **Decision Problems:** The goal is simply to determine whether at least one valid solution exists.
- **Optimization Problems:** Among all valid solutions, we search for the best possible one.
- **Enumeration Problems:** The goal is to generate every valid solution.

One of the biggest strengths of Backtracking is that it avoids unnecessary work. The moment a path becomes impossible, the algorithm immediately stops exploring that branch. This prevents wasting time searching through solutions that can never work.

That is why Backtracking is especially powerful in problems with huge search spaces, where brute force alone would become too expensive.

In practice, Backtracking is commonly used in: Sudoku solvers; Maze games; Crossword generation; Permutations and combinations; Constraint satisfaction problems; Pathfinding problems.

So in the end, Backtracking is not just about recursion. It is about intelligent exploration: trying possibilities, rejecting failures early, and continuously searching until the correct path is discovered.

2. Implementation of Backtracking

Now that we understand the idea behind Backtracking, the next question becomes: *How does the algorithm actually work?*

At its core, Backtracking follows a very repetitive—but extremely powerful—process. **The algorithm makes a choice, explores where that choice leads, checks whether the path is still valid, and if it fails, it undoes the choice and tries another one. This continues recursively until a solution is found, or every possible path has been explored.**

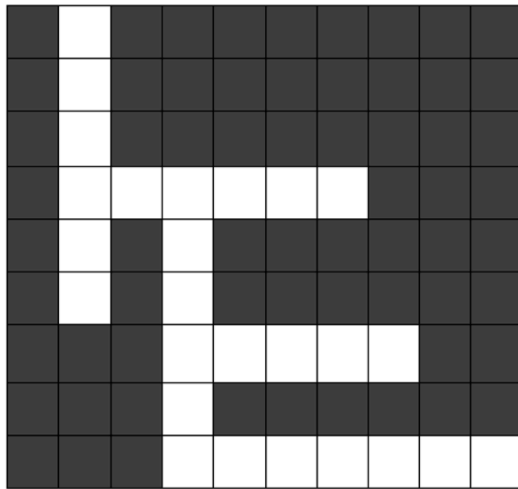
The general process looks like this:

1. Choose an initial solution or starting point
2. Explore all possible extensions of the current solution
3. If one extension works, continue exploring deeper
4. If a path becomes invalid, backtrack to the previous decision
5. Try another possibility
6. Repeat until all possible solutions are explored

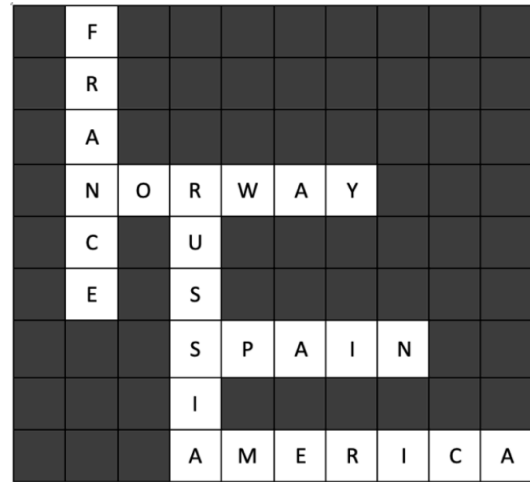
This may sound abstract at first, so let's look at two classic examples.

Crossword Game

Consider a Crossword Puzzle. A crossword consists of a grid where each box represents a letter, and words must fit together correctly through intersections.



FRANCE AMERICA SPAIN NORWAY RUSSIA



Crossword Game

The objective is simple: Find words that satisfy all clues while matching every intersecting letter correctly. Now imagine solving this manually. You might place one word into the puzzle, then attempt another word that intersects with it, then another, and another...

But suddenly, one word no longer fits. At that moment, you erase the conflicting word and try a different one. That is Backtracking. The algorithm works in almost the exact same way:

- Choose a word from the clue list
- Attempt to place it into the grid
- Check whether all intersections remain valid
- Recursively continue placing more words
- If a conflict appears: remove the previous word, backtrack, and attempt another placement

The algorithm continuously explores possible combinations of words until: the crossword is fully solved, or all possible placements have failed.

So rather than guessing randomly, Backtracking systematically searches through possibilities while rejecting invalid states early.

Maze Game

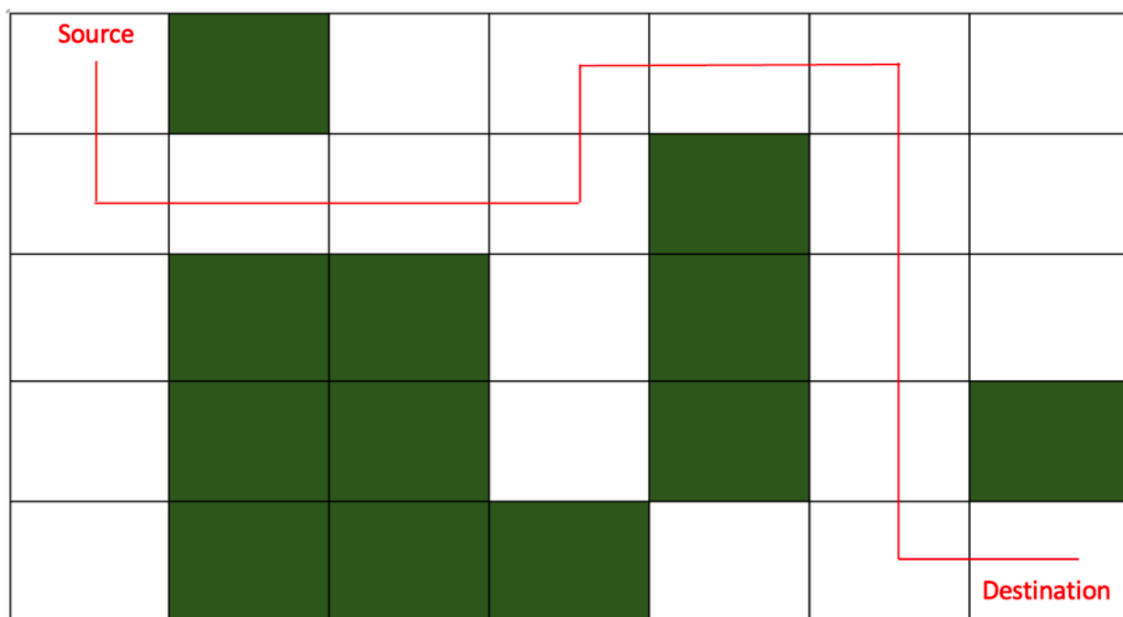
Another classic example is the Maze Game (Labyrinth Problem).

A maze is usually represented as a matrix: white cells represent open paths, colored or blocked cells represent walls. The objective is to move from the starting point to the ending point.

Using Backtracking, the algorithm begins at the start position and repeatedly tries moving in one of four directions: Up; Down; Left; Right

For every move:

- if the direction leads into a wall, the algorithm immediately rejects it, otherwise, it continues exploring recursively.
- If one path eventually reaches the exit, the algorithm returns the successful route.
- But if the path reaches a dead-end: The algorithm backtracks to the previous position, undoes the move, and tries another direction.
- This process repeats until the exit is found, or every possible route has been explored.



A solution to the maze game

The interesting part is that Backtracking does not "know" the correct direction beforehand. It simply explores possibilities until one works.

Can Backtracking Find the Shortest Path?

Technically yes. Backtracking *can* be used to search for the shortest path in a maze by exploring every possible route and comparing their lengths. However, it is usually **not the most efficient approach**.

Why? Because Backtracking does not prioritize shorter paths, may spend a long time exploring incorrect routes, and can repeatedly visit unnecessary branches before discovering the optimal solution. It might even discover a very long path first, before eventually finding the shortest one much later.

For shortest-path problems, a much better algorithm is usually: **Breadth-First Search (BFS)**. BFS explores nodes level by level, which naturally guarantees the shortest path in an unweighted maze or graph.

So while Backtracking is excellent for exploration and constraint satisfaction problems, it is not always the best choice for optimization problems involving shortest paths.

3. Applications of Backtracking

Backtracking appears in far more places than most people realize. At first glance, it may seem like a technique used only for puzzles.

But underneath, Backtracking is really about exploring possibilities while respecting constraints. And that idea appears almost everywhere in Computer Science and Mathematics. Some common applications include:

- **Mathematics:** Backtracking is heavily used in mathematical problems involving graph theory, combinatorics, permutations, combinations, and number theory. Many mathematical problems require exploring all possible arrangements while rejecting invalid configurations along the way.
- **Puzzle Solving:** This is the most recognizable use of Backtracking. Examples include: Sudoku; Crossword puzzles; N-Queens problems; Maze escape problems. These problems naturally involve trying possibilities, checking constraints, and undoing incorrect decisions. Which makes them perfect for Backtracking.
- **Scheduling Problems:** Suppose we need to assign workers to shifts, organize exam schedules, or allocate meeting times. At first, a schedule may appear valid. But later, conflicts may occur such as overlapping time

slots, unavailable resources, or impossible assignments. When this happens, the algorithm backtracks, removes conflicting assignments, and searches for alternatives.

- **Resource Allocation:** Backtracking can also allocate machines, personnel, rooms, or computational resources while satisfying certain constraints. If one allocation fails, the algorithm explores another configuration.
- **Network Optimization:** In networking problems, Backtracking may explore routing paths, network layouts, or connection configurations. If one configuration fails to satisfy optimization requirements, the algorithm returns and tests another arrangement.

4. Advantages of Backtracking

Backtracking has several important strengths:

- **Handles Multiple Solutions Well:** Backtracking is extremely useful for problems that contain many possible answers, or unknown search spaces. Instead of relying on a single path, it systematically explores possibilities.
- **Naturally Works with Recursion:** Many Backtracking problems can be divided into smaller subproblems recursively. Each recursive step represents one decision, one partial solution, or one branch of the search tree.
- **Strong Exploration Ability:** Backtracking is highly flexible because it can thoroughly search the entire solution space. This makes it powerful for constraint satisfaction problems where: missing even one possibility could matter.
- **Supports Pruning:** One of the most important optimizations is called **pruning**. Pruning means stopping exploration early when a path can no longer lead to a valid solution. Instead of wasting time exploring impossible branches, the algorithm abandons them immediately. This can drastically reduce runtime.
- **Can Be Parallelized:** In many cases, different branches of the search tree are independent. This means multiple recursive paths can sometimes run simultaneously using multiple threads, parallel systems, or distributed computation.

5. Disadvantages of Backtracking

Despite its strengths, Backtracking also has serious limitations.

- **High Time Complexity:** Backtracking can become extremely slow for large problems. Why? Because the algorithm may need to explore a massive number of possibilities, including many dead ends. In worst-case scenarios, the search space grows exponentially.
- **High Memory Usage:** Since recursive calls store partial states and decisions, Backtracking may consume large amounts of memory. Deep recursive trees can quickly become expensive.
- **Redundant Computation:** Sometimes, the algorithm repeatedly explores similar states or recalculates the same subproblems. Without optimization techniques such as memoization, caching, or pruning, performance can degrade significantly.
- **Not Ideal for Real-Time Problems:** Backtracking is generally unsuitable for real-time systems, continuous-variable problems, or situations requiring extremely fast responses. This is because the algorithm may take too long before finding a valid solution.

Summary

Backtracking is one of the most human-like problem-solving techniques in Computer Science. Because honestly, this is exactly how humans solve many difficult problems. We try something, realize it fails, return to an earlier decision, and attempt another approach. The key difference between simple trial-and-error and Backtracking is this: **Constraints**.

Backtracking does not blindly generate possibilities. It continuously checks whether the current path is still valid. The moment a constraint is violated, the algorithm immediately stops exploring that branch and returns to an earlier state. That ability to explore, reject, undo, and retry is what makes Backtracking so powerful.

Throughout this section, we saw that Backtracking builds solutions incrementally, explores possibilities recursively, and systematically searches through decision trees.

We also explored classic examples such as: Crossword puzzles; Maze traversal; Scheduling problems; Resource allocation

And while Backtracking can sometimes be expensive due to large search spaces, techniques like pruning can greatly improve efficiency by eliminating impossible paths early.

So in the end, Backtracking is not simply about recursion. It is about intelligent exploration under constraints.

References

<https://cs.lmu.edu/~ray/notes/backtracking/>

<https://www.geeksforgeeks.org/dsa/backtracking-algorithms/>

<https://www.geeksforgeeks.org/dsa/introduction-to-backtracking-2/>

<https://www.youtube.com/watch?v=DKCbsiDBN6c&t=354s>

In the following sub-sections, we will study several famous problems that are naturally solved using the Backtracking technique.

🔗 [3.1. Finding All Paths in a Graph](#)

🔗 [3.2. Sudoku Solver Problem](#)

🔗 [3.3. N-Queens Problem](#)

Next Section:

🔗 [4. Dynamic Programming](#)